

Báo cáo Lab 5 - Part 1: PyTorch

- Nguyễn Thùy Trang - 2200128

Giải thích các bước làm

1. Tạo Tensor (Task 1.1):

- Khởi tạo tensor từ list Python và NumPy array để hiểu cách chuyển đổi dữ liệu giữa các cấu trúc này.
- Tạo tensor với các giá trị hằng số (ones) và giá trị ngẫu nhiên (rand) với shape tương tự tensor đã có.
- In ra shape, dtype và device của các tensor để kiểm tra thuộc tính của chúng.

2. Các phép toán trên Tensor (Task 1.2):

- Thực hiện các phép toán cơ bản trên tensor như cộng element-wise, nhân scalar và nhân ma trận (sử dụng `.T` cho chuyển vị).
- In kết quả của từng phép toán để quan sát output.

3. Indexing và Slicing (Task 1.3):

- Sử dụng indexing và slicing để truy cập các phần tử, hàng và cột cụ thể của tensor.
- In các phần được trích xuất để xác nhận kết quả.

4. Thay đổi hình dạng Tensor (Task 1.4):

- Tạo một tensor ngẫu nhiên có kích thước 4x4.
- Sử dụng phương thức `.reshape()` (hoặc `.view()`) để biến đổi tensor gốc thành tensor có kích thước 16x1.
- In tensor gốc và tensor sau khi thay đổi hình dạng.

5. Thực hành với Autograd (Task 2.1):

- Tạo một tensor `x` và thiết lập `requires_grad=True` để theo dõi gradient.
- Thực hiện chuỗi các phép toán ($y = x + 2$, $z = y * y * 3$) để xây dựng đồ thị tính toán.
- Sử dụng `z.backward(retain_graph=True)` để tính đạo hàm của `z` theo `x`.
- In thuộc tính `.grad` của `x` để xem giá trị đạo hàm.
- Thử gọi lại `z.backward()` lần thứ hai mà không có `retain_graph=True` để chứng minh rằng đồ thị bị giải phóng sau lần backward đầu tiên và quan sát lỗi.

6. Sử dụng Lớp Linear (Task 3.1):

- Khởi tạo một lớp `torch.nn.Linear` để thực hiện phép biến đổi tuyến tính từ 5 chiều sang 2 chiều.
- Tạo một tensor đầu vào ngẫu nhiên có kích thước (3, 5) (3 mẫu, 5 đặc trưng).
- Truyền tensor đầu vào qua lớp `Linear` và in kích thước đầu vào, đầu ra và tensor đầu ra.

7. Xây dựng Mô hình Đơn giản với `nn.Module` (Task 3.2):

- Định nghĩa một lớp mô hình tùy chỉnh `MyFirstModel` kế thừa từ `nn.Module`.
- Trong phương thức `__init__`, định nghĩa các layer cần thiết: `nn.Embedding`, `nn.Linear`, `nn.ReLU`, `nn.Linear`.
- Trong phương thức `forward`, định nghĩa luồng dữ liệu đi qua các layer.
- Khởi tạo mô hình với các tham số ví dụ.
- Tạo một tensor đầu vào mẫu (ví dụ một câu với 4 từ) và truyền qua mô hình.
- In shape của output để kiểm tra kích thước đầu ra của mô hình.

Hướng dẫn thực thi

1. **Mở Notebook:** Mở file notebook trong môi trường hỗ trợ Jupyter Notebook (ví dụ: Google Colab, Jupyter Notebook cài đặt trên máy cục bộ, VS Code với extension Jupyter).
2. **Chạy các ô code.**
3. **Xem kết quả.**

Challenges and solutions:

- **Thử thách:** Hiểu rõ cơ chế hoạt động của `.backward()` và lý do tại sao nó không thể chạy lại lần thứ hai trên cùng một đồ thị mặc định.
- **Giải pháp:** Tìm hiểu về tham số `retain_graph=True` trong phương thức `.backward()` và thử nghiệm gọi `.backward()` lần thứ hai có và không có tham số này. Quan sát lỗi báo cáo khi không giữ lại đồ thị đã giúp làm rõ vấn đề.

Cite references:

- Tài liệu chính thức của PyTorch: <https://pytorch.org/docs/stable/index.html>
- Các hướng dẫn và ví dụ từ trang web PyTorch chính thức về Tensor, Autograd và `nn.Module`.

Part 1: PyTorch

```
# nếu chạy trên local thì cần tải thư viện
#! pip install torch
```

Task 1.1: Tạo tensor

```
import torch
import numpy as np
# Tạo tensor từ list
data = [[1, 2], [3, 4]]
x_data = torch.tensor(data)
print(f"Tensor từ list:\n {x_data}\n")
# Tạo tensor từ NumPy array
np_array = np.array(data)
x_np = torch.from_numpy(np_array)
print(f"Tensor từ NumPy array:\n {x_np}\n")
# Tạo tensor với các giá trị ngẫu nhiên hoặc hằng số
x_ones = torch.ones_like(x_data) # tạo tensor gồm các số 1 có cùng shape với x_data
print(f"Ones Tensor:\n {x_ones}\n")
x_rand = torch.rand_like(x_data, dtype=torch.float) # tạo tensor ngẫu nhiên
print(f"Random Tensor:\n {x_rand}\n")
# In ra shape, dtype, và device của tensor
print(f"Shape của tensor: {x_rand.shape}")
print(f"Datatype của tensor: {x_rand.dtype}")
print(f"Device lưu trữ tensor: {x_rand.device}")
```

```
Tensor từ list:
tensor([[1, 2],
        [3, 4]])
```

```
Tensor từ NumPy array:
tensor([[1, 2],
        [3, 4]])
```

```
Ones Tensor:
tensor([[1, 1],
        [1, 1]])
```

```
Random Tensor:
tensor([[0.4312, 0.9655],
        [0.8497, 0.1493]])
```

```
Shape của tensor: torch.Size([2, 2])
Datatype của tensor: torch.float32
Device lưu trữ tensor: cpu
```

Task 1.2: Các phép toán trên Tensor

```
# Cộng x_data với chính nó
x_plus_x = x_data + x_data
print(f"x_data + x_data:\n {x_plus_x}\n")

# Nhân x_data với 5
x_times_5 = x_data * 5
print(f"x_data * 5:\n {x_times_5}\n")

# Nhân ma trận x_data với x_data.T
x_matmul_x_T = x_data @ x_data.T
print(f"x_data @ x_data.T:\n {x_matmul_x_T}\n")
```

```
x_data + x_data:
tensor([[2, 4],
        [6, 8]])
```

```
x_data * 5:
```

```
tensor([[ 5, 10],
        [15, 20]])

x_data @ x_data.T:
tensor([[ 5, 11],
        [11, 25]])
```

Task 1.3: Indexing và Slicing

```
# Lấy ra hàng đầu tiên
first_row = x_data[0, :]
print(f"Hàng đầu tiên của x_data:\n {first_row}\n")

# Lấy ra cột thứ hai
second_column = x_data[:, 1]
print(f"Cột thứ hai của x_data:\n {second_column}\n")

# Lấy ra giá trị ở hàng thứ hai, cột thứ hai
element = x_data[1, 1]
print(f"Giá trị ở hàng thứ hai, cột thứ hai của x_data:\n {element}\n")
```

Hàng đầu tiên của x_data:
tensor([1, 2])

Cột thứ hai của x_data:
tensor([2, 4])

Giá trị ở hàng thứ hai, cột thứ hai của x_data:
4

Task 1.4: Thay đổi hình dạng Tensor

```
# Tạo tensor có shape (4, 4) với các giá trị ngẫu nhiên
random_tensor = torch.rand(4, 4)
print(f"Tensor gốc (4, 4):\n {random_tensor}\n")

# Biến đổi tensor thành shape (16, 1) bằng reshape/view
reshaped_tensor = random_tensor.view(16, 1)
reshaped_tensor = random_tensor.reshape(16, 1)
print(f"Tensor sau khi dùng reshape (16, 1):\n {reshaped_tensor}\n")
```

Tensor gốc (4, 4):
tensor([[0.1163, 0.5899, 0.2471, 0.6158],
 [0.1844, 0.9662, 0.8972, 0.0333],
 [0.6435, 0.7237, 0.5988, 0.2017],
 [0.3272, 0.8787, 0.2810, 0.5406]])

Tensor sau khi dùng reshape (16, 1):
tensor([[0.1163],
 [0.5899],
 [0.2471],
 [0.6158],
 [0.1844],
 [0.9662],
 [0.8972],
 [0.0333],
 [0.6435],
 [0.7237],
 [0.5988],
 [0.2017],
 [0.3272],
 [0.8787],
 [0.2810],
 [0.5406]])

Part 2: Derivate

Task 2.1: Thực hành với autograd

Câu hỏi: Chuyện gì xảy ra nếu bạn gọi `z.backward()` một lần nữa? Tại sao?

- Sau khi `backward` lần 1, nếu để chế độ mặc định thì đồ thị tính toán (lưu trữ các giá trị gradient tại các nút) sẽ bị xóa. Vì vậy nếu gọi `.backward()` lần thứ hai trên cùng một biểu thức (hoặc một tensor thuộc cùng đồ thị), thì PyTorch sẽ không thể tính lại gradient, vì các tensor trung gian (saved tensors) đã bị xóa sau lần `.backward()` đầu tiên.

```
# Tạo một tensor và yêu cầu tính đạo hàm cho nó
x = torch.ones(1, requires_grad=True)
print(f"x: {x}")
# Thực hiện một phép toán
y = x + 2
print(f"y: {y}")
# y được tạo ra từ một phép toán có x, nên nó cũng có grad_fn
print(f"grad_fn của y: {y.grad_fn}")
# Thực hiện thêm các phép toán
z = y * y * 3
# Tính đạo hàm của z theo x
z.backward(retain_graph=True) # tương đương z.backward(torch.tensor(1.))
# Đạo hàm được lưu trong thuộc tính .grad
# Ta có z = 3 * (x+2)^2 => dz/dx = 6 * (x+2). Với x=1, dz/dx = 18
print(f"Đạo hàm của z theo x: {x.grad}")
```

```
x: tensor([1.], requires_grad=True)
y: tensor([3.], grad_fn=<AddBackward0>)
grad_fn của y: <AddBackward0 object at 0x7cf721248070>
Đạo hàm của z theo x: tensor([18.])
```

```
# Thử gọi lại backward lần 2
try:
    z.backward()
    print(f"Đạo hàm lần 2 của z theo x: {x.grad}")
except Exception as e:
    print(f"Lỗi khi gọi backward lần 2: {e}")
```

Lỗi khi gọi backward lần 2: Trying to backward through the graph a second time (or directly access saved tensors after they have

Task 3:

Task 3.1

```
# Khởi tạo một lớp Linear biến đổi từ 5 chiều -> 2 chiều
linear_layer = torch.nn.Linear(in_features=5, out_features=2)
# Tạo một tensor đầu vào mẫu
input_tensor = torch.randn(3, 5) # 3 mẫu, mỗi mẫu 5 chiều
# Truyền đầu vào qua lớp linear
output = linear_layer(input_tensor)
print(f"Input shape: {input_tensor.shape}")
print(f"Output shape: {output.shape}")
print(f"Output:\n {output}")
```

```
Input shape: torch.Size([3, 5])
Output shape: torch.Size([3, 2])
Output:
tensor([[ -0.1809, -0.2548],
        [ -0.0717,  0.2291],
        [ 1.0195,  2.0895]], grad_fn=<AddmmBackward0>)
```

Task 3.2

```
from torch import nn
class MyFirstModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim):
        super(MyFirstModel, self).__init__()
        # Định nghĩa các lớp (layer) bạn sẽ dùng
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.linear = nn.Linear(embedding_dim, hidden_dim)
        self.activation = nn.ReLU() # Hàm kích hoạt
        self.output_layer = nn.Linear(hidden_dim, output_dim)
    def forward(self, indices):
```

```
# Định nghĩa luồng dữ liệu đi qua các lớp

# 1. Lấy embedding
embeds = self.embedding(indices)
# 2. Truyền qua lớp linear và hàm kích hoạt
hidden = self.activation(self.linear(embeds))
# 3. Truyền qua lớp output
output = self.output_layer(hidden)
return output

# Khởi tạo và kiểm tra mô hình
model = MyFirstModel(vocab_size=100, embedding_dim=16, hidden_dim=8, output_dim=2)
input_data = torch.LongTensor([[1, 2, 5, 9]]) # một câu gồm 4 từ
output_data = model(input_data)
print(f"Model output shape: {output_data.shape}")
```

```
Model output shape: torch.Size([1, 4, 2])
```